

Assessing Polygenic Risk for Idiopathic Pulmonary Fibrosis Using GWAS-Based Modeling Strategies

Team Members: Lauren Urban, Tada Vargas, Helen Wolde, Yifan Zhao

May 16, 2025

Introduction:

Idiopathic Pulmonary Fibrosis (IPF) is a complex, chronic, progressive lung disease of unknown etiology. It is characterized by fibrotic scarring of the lung tissue, with a median survival time of approximately three years following diagnosis. It affects an estimated 50,000 individuals annually in the United States, and this number is expected to rise as the population continues to age. While the disease often presents sporadically, 15–25% of individuals with IPF report a family history of idiopathic interstitial pneumonia, suggesting a heritable component to the disease.

Over the past decade, genome-wide association studies (GWAS) have provided significant insights into the genetic architecture of IPF. Major studies by Fingerlin et al. (2013) and others have identified both common and rare genetic variants associated with increased risk of IPF. Most notably, the MUC5B promoter variant is the most robust and consistently replicated genetic risk factor, while additional loci such as TERT, TERC, FAM13A, DSP, OBFC1, ATP11A, and DPP9 implicate biological pathways related to mucosal immunity, DNA repair, and epithelial cell function.

Polygenic Risk Scores (PRS) provide a method of translating GWAS findings into a single, interpretable metric of genetic risk by aggregating the effects of many common variants with modest individual contributions. These scores offer a promising approach for improving our

understanding of genetic susceptibility to complex diseases like IPF and may assist in early risk diagnosis and future clinical decision making.

In this project, we aim to use individual-level GWAS data from a cohort of participants with European ancestry to estimate and validate polygenic risk scores for IPF. We evaluate the predictive performance of different PRS models, including a simple model using all SNPs and a pruning and thresholding approach. Our goal is to determine whether incorporating established genetic loci can improve risk prediction, and to assess the practical utility of PRS in capturing the genetic liability of IPF.

Methods

Study Population and Genotyping

We analyzed individual level genome-wide genotype data from unrelated adults of European ancestry (see Table 1 for sample characteristics). All cases met the formal criteria for IPF and controls had no history of interstitial lung disease. DNA was extracted from samples and genotyped onto arrays for future analysis.

Data Management & Quality Control

The dataset underwent extensive quality control prior to delivery, including the removal of individuals with discordant reported and genetically inferred sex, non-European ancestry, second-degree or closer relatives, duplicate samples, and those with more than 1% missing genotypes. Variants were filtered based on call rate ($\geq 95\%$), differential missingness between cases and controls, and Hardy–Weinberg equilibrium in controls ($P > 1 \times 10^{-4}$). Variants were then split into common ($MAF > 3\%$) and rare ($MAF < 3\%$) sets, with only common variants retained for analysis.

Upon receiving the dataset, we performed an additional round of SNP-level filtering, removing variants with more than 2% missingness. This excluded 42 SNPs, resulting in a final dataset of 6,465 common variants. No additional individuals were removed. To inspect the quality of the data and confirm its suitability for PRS modeling, we conducted a GWAS adjusting for sex and evaluated the distribution of resulting p-values using the GWASTools (Gogarten, 2012) and snpStats (Clayton, 2022) R packages.

Polygenic Risk Score Construction

Two complementary approaches were used to construct individual-level PRS: one based on marginal GWAS signals, and the other on penalized joint modeling of SNPs using individual-level genotype data. Both approaches assumed additive genetic effects and were implemented within a common model development framework involving training, cross-validation, and testing (Kuhn, Johnson, 2013). The dataset was first split into an 80% training set and a 20% testing set. Within the training set, 5-fold cross-validation was used to tune model parameters and select the best-performing PRS. Final evaluation of predictive performance was carried out in the held-out testing set. Sex was included as a covariate in both models, given known sex differences in IPF risk. Missing genotypes were imputed using SNP-wise means: by default in PLINK (Purcell et al., 2007) for Model 1, and using means from the training set for Model 2.

In Model 1, clumping and thresholding was implemented in PLINK (Purcell et al., 2007) by first applying LD-based clumping to the IPF summary statistics, retaining only the most significant SNP in each LD block at r^2 cutoffs of 0.1, 0.2, and 0.3. From each clumped set, we generated polygenic scores at p-value thresholds (p_i) of 0.01, 0.05, 0.1 and 0.3; producing 12 candidate PRS in total. We then performed a single 5-fold cross-validation pass within the training data to

compute each score's mean AUC across the folds. The p_r/r^2 combination that maximized this mean AUC was selected as optimal, and the corresponding PRS was evaluated in the hold-out fold to quantify final predictive performance. Due to software limitations, two versions of the PRS were constructed based on the testing data to evaluate the predictive performance of Model 1. The first version, referred to as the unadjusted PRS, was calculated using PLINK's `--score` function (Purcell et al., 2007), which applied log-transformed SNP-level effect estimates from the GWAS to the genotype data without accounting for sex. The second version, the adjusted PRS, incorporated the effect of sex by adding the average log-odds ratio for sex—estimated across all selected SNPs—to the unadjusted PRS scores. Both versions were evaluated using the AUC to assess predictive performance.

In Model 2, we used penalized logistic regression (Friedman, Hastie, & Tibshirani, 2010) via the `glmnet` package, with SNP dosages and sex as predictors. To reduce computational burden, we restricted the input to the top 2,000 most variable SNPs (based on standard deviation) across the training set. Within the training data, 5-fold cross-validation was used to select the optimal penalty parameter λ by minimizing cross-validated deviance. We fit three models corresponding to $\alpha = 1$ (LASSO), $\alpha = 0.5$ (Elastic Net), and $\alpha = 0$ (Ridge). For each α , the model was refit on the full training set using the selected λ , and predicted probabilities were generated for individuals in the testing set. ROC curves were constructed using the `pROC` package (Robin et al., 2011), and the AUC was used to evaluate and compare predictive performance across the three regularization schemes. Although sex was not explicitly specified as an unpenalized covariate, it was consistently retained in all final models.

Results

Demographic Information

Table 1 presents demographics of the study cohort. Among the 3,617 IPF cases and 4,442 controls, males made up a slightly larger proportion of cases (71.4%) compared to controls (41.2%), while smoking status was fully missing in all controls. The family history of IPF was more common in cases (13.2%) than in controls (6.1%).

	Case (N=3617)	Control (N=4442)	Overall (N=8059)
Sex			
Female	1033 (28.6%)	2610 (58.8%)	3643 (45.2%)
Male	2584 (71.4%)	1832 (41.2%)	4416 (54.8%)
Ever Smoked			
no	944 (26.1%)	0 (0%)	944 (11.7%)
yes	2111 (58.4%)	0 (0%)	2111 (26.2%)
Missing	562 (15.5%)	4442 (100%)	5004 (62.1%)
IPF Family History			
0	3139 (86.8%)	4173 (93.9%)	7312 (90.7%)
1	478 (13.2%)	269 (6.1%)	747 (9.3%)

Table 1. Summary of IPF Overall Demographics

Genetic Architecture

To assess whether the dataset was suitable for PRS modeling, we first examined the p-value distribution from a GWAS adjusting for sex. As shown in Figure 1, the Manhattan plot displays – $\log_{10}(p)$ values for SNPs across chromosomes. A number of SNPs exceed the conventional genome-wide significance threshold ($P < 5 \times 10^{-8}$), indicated by the dashed horizontal line. Notably, chromosome 11 shows strong signals with many significant SNPs, some of which are truncated due to the y-axis limit. These findings suggest that the genetic architecture of the trait may be polygenic and that the dataset is well-suited for downstream PRS analysis.

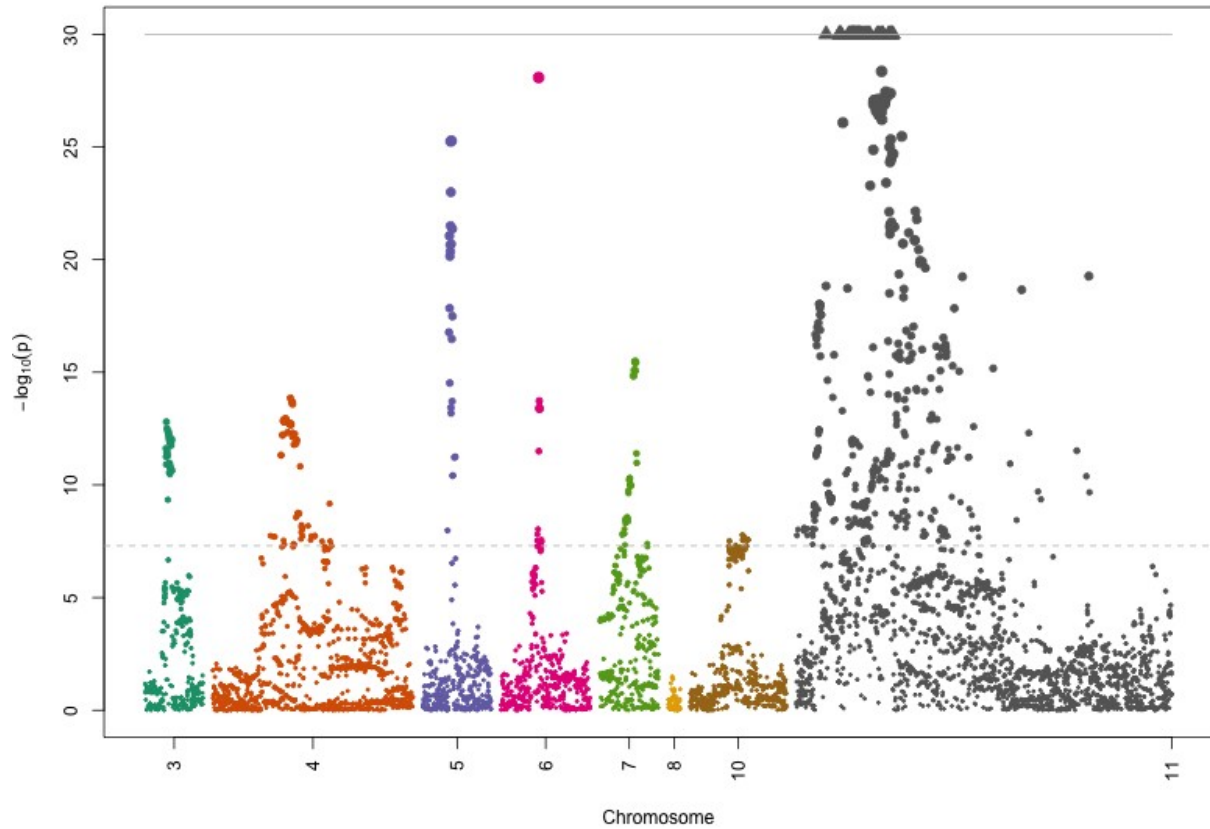


Figure 1. Manhattan Plot: GWAS analysis adjusting for Sex

Parameter Tuning

As part of the parameter tuning process for Model 1, we evaluated 12 candidate PRS defined by combinations of four p-value thresholds (0.01, 0.05, 0.10, 0.30) and three LD clumping thresholds ($r^2 = 0.10, 0.20, 0.30$). As shown in Table 2, each cell represents the mean AUC obtained from 5-fold cross-validation within the training set. The results show minimal variation in AUC across the parameter grid, with all combinations yielding values between 0.727 and 0.729. The highest mean AUC (0.729) was observed consistently across all p-value thresholds when $r^2 = 0.10$. Based on these results, we selected the combination of $p_t = 0.01$ and $r^2 = 0.10$ for final model evaluation in the testing set. We also examined the average number of SNPs selected using each combination

of parameters, which shows that the number of selected SNPs only vary with r^2 , but not p_t . See Table S1 in the supplement for details.

P-value threshold	$r^2 = 0.10$	$r^2 = 0.20$	$r^2 = 0.30$
0.01	0.729	0.728	0.727
0.05	0.729	0.728	0.727
0.10	0.729	0.728	0.727
0.30	0.729	0.728	0.727

Table 2. Mean 5-fold AUC for Clumping and Thresholding Model (Model 1)

For Model 2, the tuning parameter was chosen as the lambda that minimizes the cross-validation deviance. The chosen λ s are recorded in Table 3, along with the AUC.

Model	$\lambda_{\min}$	AUC
Lasso ($\alpha = 1$)	0.0048	0.772
Elastic Net ($\alpha = 0.5$)	0.0105	0.770
Ridge ($\alpha = 0$)	0.0500	0.755

Table 3. Mean 5-fold AUC for Penalized Regression Models (Model 2)

Model Evaluation

We evaluated the ability of a PRS to discriminate IPF cases from controls using clumping and thresholding (Model 1) and three penalized regression approaches (Model 2: Lasso, Ridge, and Elastic Net). As shown in **Figure 2**, Model 1 achieved an AUC of 0.724 when unadjusted and 0.761 after incorporating sex, indicating that sex is a meaningful covariate that improves discriminative performance. Elastic Net—the top performer among Model 2 methods—reached

an AUC of 0.77. Overall, all models demonstrated good discriminatory ability, suggesting that genetic risk scores can effectively distinguish IPF cases from controls in this dataset.

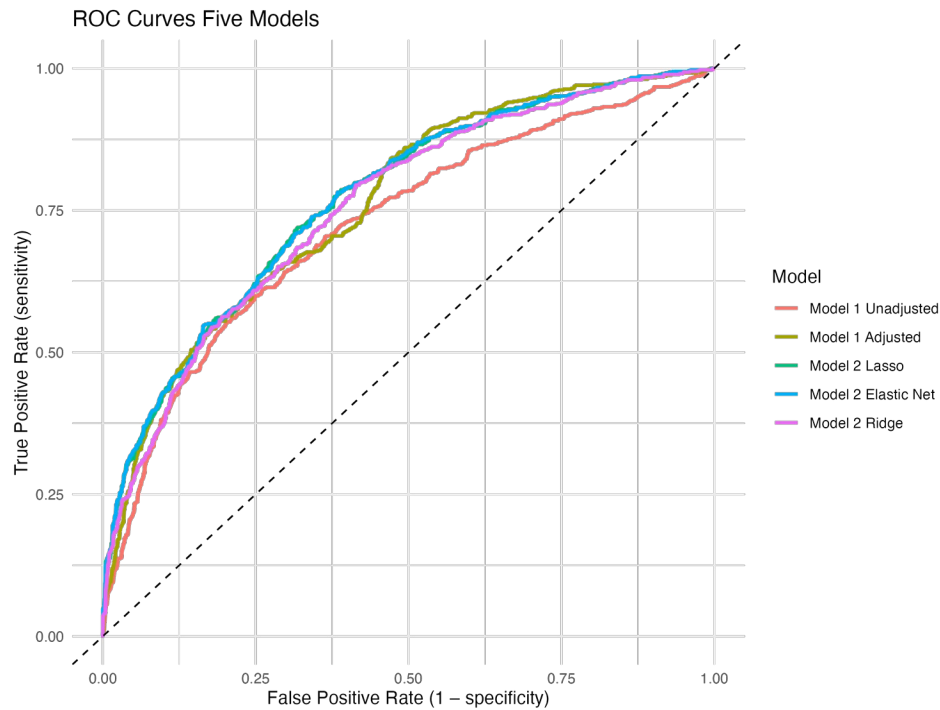
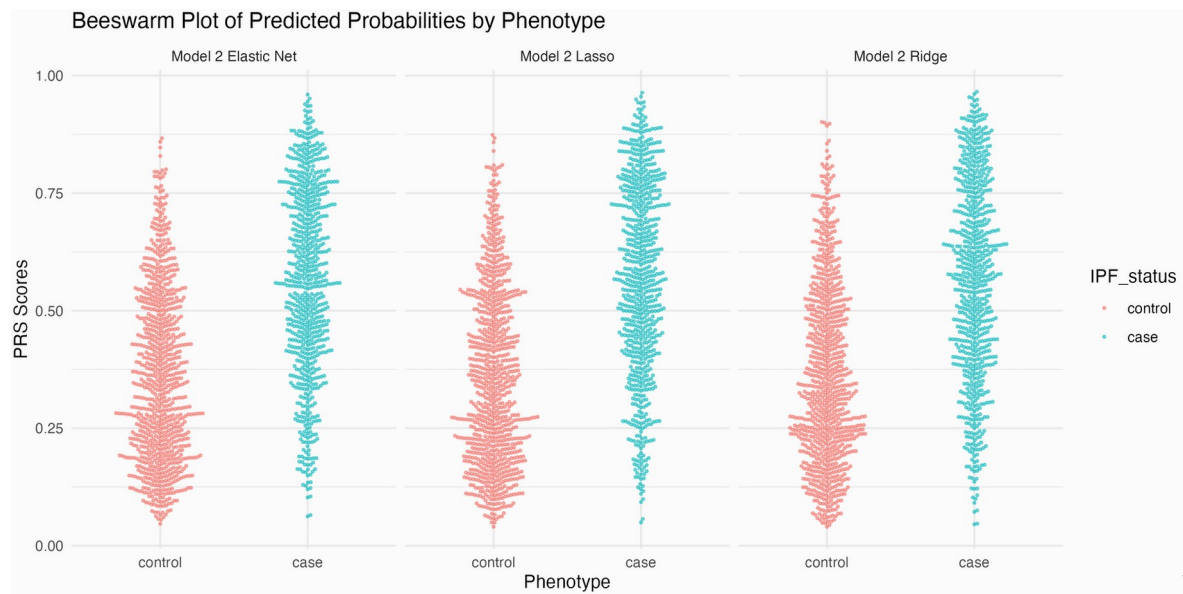


Figure 2. ROC curves comparing PRS scores for the Five Models

Figure 3 presents a Beeswarm plot of predicted scores that is aimed at visualizing how well Model 2 separated cases and controls. Overall, we see a higher score in cases than controls for all the three penalized models, indicating each method captures IPF-related genetic signal. Particularly, Elastic Net produced scores that were more clearly separated, with cases clustering near the top and controls near the bottom of the distribution. This visual separation aligns with the AUC rankings observed in the ROC analysis.



Figur

e 3. Beeswarm Plot of Predicted Probabilities by Phenotype

Figure 4 presents Euler plots comparing the sets of SNPs retained by Model 1 (marginal clumping-based PRS) and Model 2 (penalized regression approaches: Lasso and Elastic Net). Ridge regression was excluded from this comparison because it does not perform variable selection. Since the two clumped PRS models from Model 1 retained identical SNP sets, only one was included in the plot. We also compared these findings with the seven SNPs previously reported by Fingerlin et al., but none overlapped with our results. While we anticipated substantial agreement between the marginal and penalized models, the Euler plots revealed limited overlap: only a small subset of SNPs was shared between Lasso and Elastic Net, and there was almost no intersection with Model 1. We also examined the proportion of overlap between models within the subsets of risk SNPs (those with positive log-odds ratios, shown in Panel a) and protective SNPs (with negative log-odds ratios, shown in Panel c). In both cases, the pattern of limited overlap remained consistent.

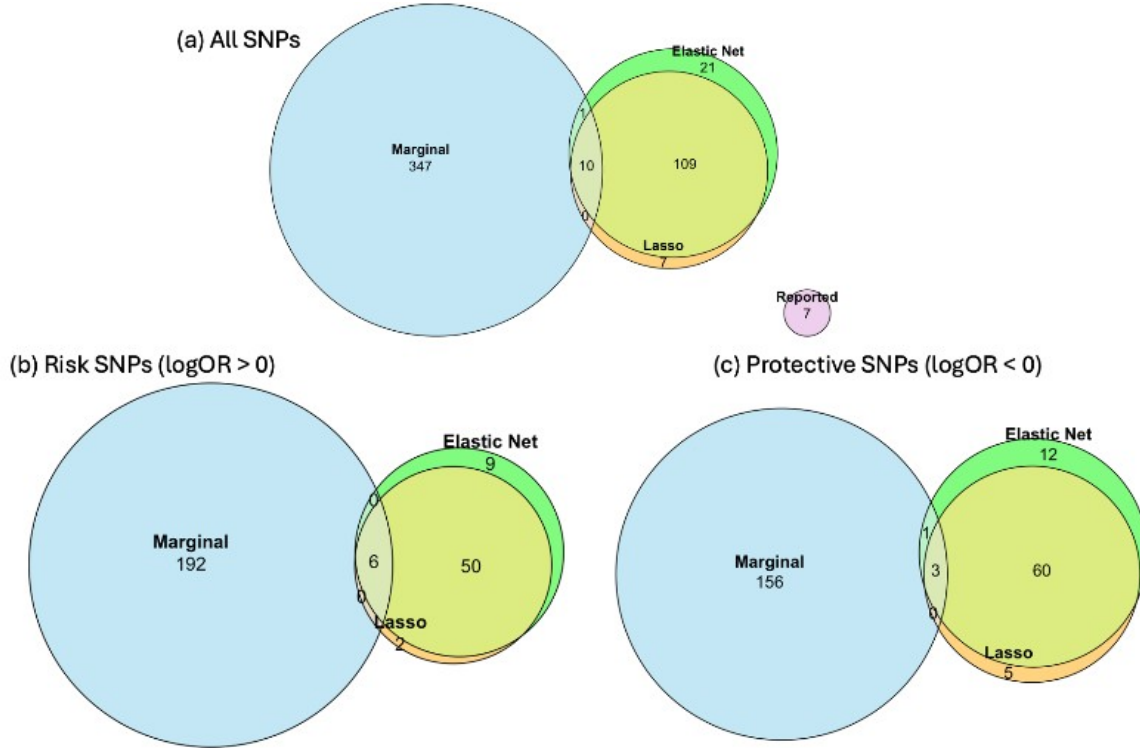


Figure 4. Euler plots comparing selected SNPs from Model 1 (marginal), Lasso, and Elastic Net models. (a) All SNPs (b) Risk SNPs ($\log OR > 0$) (c) Protective SNPs ($\log OR < 0$)

Discussion & Conclusions

Our analysis confirms that PRS models can meaningfully predict an individual's likelihood of developing IPF. We compared two modeling strategies: a marginal approach using LD-based clumping and thresholding, and several regularized regression models, including Lasso, Ridge, and Elastic Net. All models demonstrated similarly strong discrimination ($AUC \approx 0.76$). While Elastic Net showed marginally better separation between cases and controls, the overall differences between modeling strategies were modest and all performed well, regardless of whether sex was included as a covariate.

Despite similar predictive accuracy, the SNPs retained by the marginal and penalized models differed substantially, with some overlap between Lasso and Elastic Net, and almost none with the clumped model. This likely reflects the distinct feature selection mechanisms used by each approach and highlights how modeling choices and genetic architecture influence variant prioritization. We also found no overlap between our selected variants and those reported by Fingerlin et al., which may be attributed to differences in study populations, methodological approaches, or SNP inclusion criteria.

This analysis has several limitations. First, we focused only on SNPs selected by the clumped model, Lasso, and Elastic Net. Although Ridge was included in predictive evaluation, it was excluded from SNP-level comparison since it does not perform variable selection. Second, our models were trained and evaluated using a single train-test split. While this is common in early-stage analyses, it may yield optimistic performance estimates and does not account for variability across different data subsets. Incorporating k-fold cross-validation or external replication would provide a more robust assessment of model generalizability.

Method-specific caveats further qualify our findings. The clumping and thresholding approach evaluates each SNP independently and ignores joint effects among correlated variants, which can inflate the importance of index SNPs. Lasso tends to select one SNP per LD block, simplifying interpretation but possibly omitting relevant variants. Ridge retains all predictors with heavy shrinkage, reducing interpretability despite offering desirable statistical properties. Elastic Net combines Lasso and Ridge penalties, accommodating LD structure more effectively, though its performance depends on careful tuning and is computationally more intensive.

In summary, penalized regression models—particularly Elastic Net—offer a viable alternative to clumping-based PRS approaches. While all models showed strong predictive performance,

differences in selected SNPs and model assumptions suggest that integrating multiple methods and applying more robust validation strategies may enhance future risk prediction efforts.

References

- Clayton, D. (2022). *snpStats: SnpMatrix and XSnpMatrix classes and methods* (Version 1.57.0) [R package]. Bioconductor. <https://bioconductor.org/packages/snpStats>
- Fingerlin, T. E., Zhang, W., Yang, I. V., Ainsworth, H. C., Russell, P. H., Blumhagen, R. Z., Wagner, M. P., Carr, J. L., Brown, K. K., Lynch, D., Schwarz, M. I., Steele, M. P., Murphy, E., Groshong, S. D., Cosgrove, G. P., Allen, R. J., Ellison, M. C., Markin, C. R., Woodruff, P. G., ... Schwartz, D. A. (2013). Genome-wide association study identifies multiple susceptibility loci for pulmonary fibrosis. *Nature Genetics*, 45(6), 613–620. <https://doi.org/10.1038/ng.2609>
- Friedman, J., Hastie, T., & Tibshirani, R. (2010). Regularization paths for generalized linear models via coordinate descent. *Journal of Statistical Software*, 33(1), 1-22. <https://doi.org/10.18637/jss.v033.i01>
- Gogarten, S. M., Bhangale, T., Conomos, M. P., Laurie, C. A., McHugh, C. P., Painter, I., ... Laurie, C. C. (2012). *GWASTools: An R/Bioconductor package for quality control and analysis of genome-wide association studies*. *Bioinformatics*, 28(24), 3329–3331. <https://doi.org/10.1093/bioinformatics/bts610>
- Kuhn, M., & Johnson, K. (2013). *Applied predictive modeling* (Fig. 4.4). Springer-Verlag. <https://doi.org/10.1007/978-1-4614-6849-3>
- Purcell, S., Neale, B., Todd-Brown, K., Thomas, L., Ferreira, M. A. R., Bender, D., ... Sham, P. C. (2007). PLINK: A tool set for whole-genome association and population-based linkage analyses. *American Journal of Human Genetics*, 81(3), 559-575. <https://doi.org/10.1086/519795>
- Robin, X., Turck, N., Hainard, A., Tiberti, N., Lisacek, F., Sanchez, J.-C., & Müller, M. (2011). pROC: An open-source package for R and S+ to analyze and compare ROC curves. *BMC Bioinformatics*, 12, 77. <https://doi.org/10.1186/1471-2105-12-77>

Appendix

Supplement

P-value	$r^2=0.1$	$r^2=0.2$	$r^2=0.3$
0.3	353.2	495	622.4
0.1	353.2	495	622.4
0.05	353.2	495	622.4
0.01	353.2	495	622.4

Table S1. Mean Number of SNPs for Each Combination of p_t and r^2 .

Code Appendix

Data Filtering

All SNPs with more than 2% missing values were filtered out. The filtered data was saved in the *ProcessedData* folder.

```
plink --bfile RawData/prs_common_group1 \
      --geno 0.02 \
      --make-bed \
      --out ProcessedData/prs_common_group1_filtered
```

Table 1

This is the code to produce Table 1 in the report.

```
# Read in the pheno data
pheno <- read.csv(here("RawData", "phenotype.csv"),
                 header = TRUE)

# Ensure categorical variables are factors
pheno$phenotype <- factor(pheno$phenotype, levels = c("case", "control"),
                          labels = c("Case", "Control"))
pheno$Phenotype <- factor(pheno$phenotype)
pheno$Ever_Smoke <- factor(pheno$ever_smoking)
pheno$IPF_Family_History <- factor(pheno$ipf_family_history)
pheno$Sex <- factor(pheno$SEX)

# Define variables to summarize
vars <- c("Sex", "Ever_Smoke", "IPF_Family_History")

label(pheno$Sex) <- "Sex"
label(pheno$Ever_Smoke) <- "Ever Smoked"
label(pheno$IPF_Family_History) <- "IPF Family History"
label(pheno$Phenotype) <- "Phenotype"

table1(~ Sex + Ever_Smoke + IPF_Family_History | factor(Phenotype), data=pheno)
```

GWAS

We performed a GWAS analysis adjusting for sex. Here we produce the manhattan plot and the number of SNPs with p-values below a certain threshold.

```
library(GWASTools)
library(here)
```

```

library(snpStats)
library(SNPRelate)

bfile <- read.plink(here("ProcessedData", "prs_common_group1_filtered.bed"),
                  here("ProcessedData", "prs_common_group1_filtered.bim"),
                  here("ProcessedData", "prs_common_group1_filtered.fam"))

# Convert data from SnpMatrix format to a numeric matrix
mat <- t(as(bfile$genotypes, "numeric"))

# Create the geno matrix
mgr <- MatrixGenotypeReader(genotype=mat,
                             chromosome=bfile$map$chromosome,
                             position=bfile$map$position,
                             scanID=seq(1, ncol(mat)),
                             snpID=seq(1, nrow(mat)))

# Create the SNP annotation
snpAnnot <- SnpAnnotationDataFrame(
  data = data.frame(snpID=1:nrow(mat),
                    chromosome = bfile$map$chromosome,
                    position = bfile$map$position,
                    rsID = bfile$map$snp.name,
                    alleleA = bfile$map$allele.1,
                    alleleB=bfile$map$allele.2))

# Create the sample annotation
scanAnnot <- ScanAnnotationDataFrame(
  data=data.frame(scanID = seq(1, ncol(mat)),
                  sampleID = rownames(bfile$genotypes),
                  sex = ifelse(bfile$fam$sex == 1, "M", "F"),
                  phenotype = bfile$fam$affected - 1))

# Combine into a final GenotypeData object
assocData <- GenotypeData(mgr, scanAnnot=scanAnnot, snpAnnot=snpAnnot)

# Test for association
assoc_res <- assocRegression(assocData,
                             outcome = "phenotype",
                             covar = c("sex"),
                             model.type = "logistic",
                             gene.action = "additive")

png(here("Results", "GWAS_manhattan.png"), width = 800, height = 600)
# Manhattan plot
manhattanPlot(assoc_res$Wald.pval,
               chromosome = assoc_res$chr, ylim = c(0, 30))
dev.off)

```

```

p_range <- 10^seq(-1, -10, by = -1)

p_value <- data.frame(
  p = p_range,
  count = sapply(p_range, function(p) sum(assoc_res$Wald.pval > p))
)
write.csv(p_value,
  here("Results", "p_value_count.csv"),
  row.names = FALSE)

```

Data Splitting

QC-ed data was stored in ProcessedData. From now on, the processed data will be stored in the folder ProcessedData2.

```

# Read in .fam file to get individual IDs
fam <- read.table(here("ProcessedData", "prs_common_group1_filtered.fam"))

# Set seed for reproducibility
set.seed(6655)

# Split the data into training (80%) and testing (20%) sets
train_indices <- sample(1:nrow(fam), size = 0.8 * nrow(fam))

train_fam <- fam[train_indices, 1:2]

write.table(train_fam,
  here("ProcessedData2", "training_indivs.txt"),
  quote = FALSE,
  row.names = FALSE,
  col.names = FALSE)

```

Write the plink files.

```

cd ProcessedData2

plink --bfile ../ProcessedData/prs_common_group1_filtered \
--keep training_indivs.txt \
--make-bed \
--out training_indivs

plink --bfile ../ProcessedData/prs_common_group1_filtered \
--remove training_indivs.txt \
--make-bed \
--out testing_indivs

```

Model 1: Clumping and Thresholding

Split the Training Data Into Five Folds

We will further split the training data into 5 folds for cross-validation.

```
set.seed(6655)

# Read in the training data
train_fam <- read.table(here("ProcessedData2", "training_indivs.fam"))

# Create a vector of fold numbers
folds <- sample(rep(1:5, length.out = nrow(train_fam)))
table(folds)

# Save each fold to a separate file
for (i in 1:5) {
  fold_indices <- which(folds == i)
  fold_fam <- train_fam[fold_indices, 1:2]

  write.table(fold_fam,
    here("ProcessedData2", paste0("fold_", i, ".txt")),
    quote = FALSE,
    row.names = FALSE,
    col.names = FALSE)
}
```

For each fold, we want to write a subset of plink files (both training and testing) to a new folder.

```
for i in {1..5}
do
  mkdir fold_$i
  # Make training set
  plink --bfile training_indivs \
  --remove fold_$i.txt \
  --make-bed \
  --out fold_$i/training_indivs

  # Make testing set
  plink --bfile training_indivs \
  --keep fold_$i.txt \
  --make-bed \
  --out fold_$i/testing_indivs
done
```

We also want to create a phenotype file that is compatible with plink. In plink, the binary phenotypes must be coded as 1 = control and 2 = case - not 0 and 1.

```
# Col1: pedigree, Col2: member, Col6: affected
write.table(train_fam[, c(1, 2, 6)],
            here("ProcessedData2", "training_indivs_phen.txt"),
            quote = FALSE,
            row.names = FALSE,
            col.names = FALSE)
```

We also want to create a covariate file to adjust for. The covariate file must be a plain text file with FID and IID as the first two columns, followed by any number of covariate columns. Here we want to control for sex only.

```
# In the covariate file, we want 1 to be female (reference) and 2 to be male.
pheno_train <- train_fam
pheno_train[,5] <- ifelse(pheno_train[,5] == 1, 2, 1)
# Write a covariate file
write.table(train_fam[, c(1, 2, 5)],
            here("ProcessedData2", "training_indivs_cov.txt"),
            quote = FALSE,
            row.names = FALSE,
            col.names = FALSE)
```

CV for Parameter Tuning

First, estimate univariate linear model in plink.

```
for i in {1..5}
do
  plink --bfile fold_$i/training_indivs \
        --pheno training_indivs_phen.txt \
        --covar training_indivs_cov.txt \
        --logistic \
        --out fold_$i/logistic_model
done
```

Build clumped models.

```
# Define the range of pT and r^2 values
pT_values=(0.01 0.05 0.1 0.3)
r2_values=(0.1 0.2 0.3)

# Loop through each fold
for i in {1..5}
do
  # Loop through each pT value
  for pT in "${pT_values[@]}"
  do
    # Loop through each r^2 value
    for r2 in "${r2_values[@]}"
    do
      plink --bfile fold_$i/training_indivs \
```

```

--clump fold_$i/logistic_model.assoc.logistic \
--clump-p1 $pT \
--clump-r2 $r2 \
--clump-kb 250 \
--out fold_$i/clumped_model_pT${pT}_r2${r2} \
--seed 6655
done
done
done

```

Test the models on the testing dataset in each fold.

```

# Define the range of pT and r^2 values
folds <- 1:5
pT_values <- c(0.01, 0.05, 0.1, 0.3)
r2_values <- c(0.1, 0.2, 0.3)

# Convert the OR to logOR
for (i in folds) {
  for (pt in pT_values) {
    for (r2 in r2_values) {

      folder <- paste0("ProcessedData2/fold_", i)

      # File paths
      assoc_path <- here(folder, paste0("logistic_model.assoc.logistic"))
      clumped_path <- here(folder, paste0("clumped_model_pT",
                                           pt, "_r2", r2, ".clumped"))
      output_path <- here(folder, paste0("score_input_pT", pt, "_r2", r2, ".txt"))

      # Read in the clumped model
      clumped_model <- read.table(clumped_path, header = TRUE)
      assoc_model <- read.table(assoc_path, header = TRUE)

      # Keep only rows where TEST == "ADD"
      assoc_model <- assoc_model[assoc_model$TEST == "ADD", ]

      # Subset
      common_snps <- assoc_model[assoc_model$SNP %in% clumped_model$SNP,
                                c("SNP", "A1", "OR")]

      # Calculate the logOR
      common_snps$OR <- log(common_snps$OR)

      # Save the merged data
      write.table(common_snps,
                  output_path,
                  quote = FALSE,

```

```

        row.names = FALSE,
        col.names = FALSE)
    }
}

```

Now, we can score the testing dataset using the score file we just created. The scores are unadjusted for sex, but are still good enough to calculate AUC, I assume.

```

for i in {1..5}
do
  for pT in 0.01 0.05 0.1 0.3
  do
    for r2 in 0.1 0.2 0.3
    do
      plink --bfile fold_${i}/testing_indivs \
        --score fold_${i}/score_input_pT${pT}_r2${r2}.txt 1 2 3 \
        --out fold_${i}/preds_pT${pT}_r2${r2}
    done
  done
done

```

Calculate AUC for each fold and each model.

```

# Load phenotypes
bfile_train <- read.plink(here("ProcessedData2", "training_indivs.bed"),
  here("ProcessedData2", "training_indivs.bim"),
  here("ProcessedData2", "training_indivs.fam"))
pheno_train <- bfile_train$fam
pheno_train$y <- pheno_train$affected -1

# Store AUC values
auc_values <- data.frame(pT = numeric(), r2 = numeric(),
  fold = integer(), AUC = numeric())

# Loop through each fold
for (i in 1:5) {
  for (pT in pT_values) {
    for (r2 in r2_values) {

      # File paths
      preds_path <- here("ProcessedData2", paste0("fold_", i),
        paste0("preds_pT", pT, "_r2", r2, ".profile"))

      # Read in the predictions
      preds <- read.table(preds_path, header = TRUE)

      # Merge with the training phenotypes
      merged_data <- merge(preds, pheno_train,

```

```

        by.x = "IID", by.y = "member")

# Calculate AUC
roc_obj <- roc(merged_data$y, merged_data$SCORE)
auc_val <- auc(roc_obj)

# Store the results
auc_values <- rbind(auc_values,
                    data.frame(pT = pT, r2 = r2,
                               fold = i, AUC = auc_val))
    }
}

# Save the AUC values
write.csv(auc_values,
          here("Results", "AUC_values_Training.csv"),
          row.names = FALSE)

# Calculate average AUC across each fold
avg_auc <- aggregate(AUC ~ pT + r2, data = auc_values, FUN = mean)
avg_auc <- avg_auc[order(avg_auc$AUC, decreasing = TRUE), ]
avg_auc

# Convert to wider format
auc_wide <- avg_auc %>%
  pivot_wider(names_from = r2, values_from = AUC, names_prefix = "r2_") %>%
  arrange(desc(pT))
auc_wide

# Save wide format
write.csv(auc_wide,
          here("Results", "AUC_values_Training_wide.csv"),
          row.names = FALSE)

```

Re-fit the Model Using the Best Combination of pT and r²

```

# Define the best combination of pT and r^2
pT=0.01
r2=0.1
mkdir training_indivs
# Raw model
plink --bfile training_indivs \
  --pheno training_indivs_phen.txt \
  --covar training_indivs_cov.txt \
  --logistic \
  --out training_indivs/logistic_model_final

# Clumped model
plink --bfile training_indivs \

```

```

--clump training_indivs/logistic_model_final.assoc.logistic \
--clump-p1 $pT \
--clump-r2 $r2 \
--clump-kb 250 \
--out training_indivs/clumped_model_final \
--seed 6655

# Read in the clumped model
clumped_model_final <- read.table(
  here("ProcessedData2", "training_indivs",
        "clumped_model_final.clumped"),
  header = TRUE)

# Keep only rows where TEST == "ADD"
assoc_model_final <- read.table(
  here("ProcessedData2", "training_indivs",
        "logistic_model_final.assoc.logistic"),
  header = TRUE)
assoc_model_final <- assoc_model_final[assoc_model_final$TEST == "ADD", ]

# Subset
common_snps_final <- assoc_model_final[
  assoc_model_final$SNP %in% clumped_model_final$SNP,
  c("SNP", "A1", "OR")]

# Calculate the logOR
common_snps_final$OR <- log(common_snps_final$OR)

# Save the merged data
write.table(
  common_snps_final,
  here("ProcessedData2", "training_indivs",
        "score_input_final.txt"),
  quote = FALSE,
  row.names = FALSE,
  col.names = FALSE)

```

Unadjusted PRS

```
mkdir testing_indivs
```

```

# Test the model on the testing dataset
plink --bfile testing_indivs \
--score training_indivs/score_input_final.txt 1 2 3 \
--out testing_indivs/preds_final

```

Calculate the unadjusted AUC.

```

# Read in the test data
bfile_test <- read.plink(
  here("ProcessedData2", "testing_indivs.bed"),
  here("ProcessedData2", "testing_indivs.bim"),
  here("ProcessedData2", "testing_indivs.fam"))
pheno_test <- bfile_test$fam
pheno_test$y <- pheno_test$affected - 1

```

```

# Read in the predictions
preds_final <- read.table(here("ProcessedData2", "testing_indivs",
                             "preds_final.profile"), header = TRUE)
# Merge with the test phenotypes
merged_data_final <- merge(preds_final, pheno_test,
                           by.x = "IID", by.y = "member")
# Calculate AUC
roc_obj_final <- roc(merged_data_final$y, merged_data_final$SCORE)
auc_val_final <- auc(roc_obj_final)
auc_val_final

# Save roc object
saveRDS(roc_obj_final,
        file = here("Results", "roc_obj_final_unadjusted.rds"))

```

Now, save the effects of the SNPs in different files.

Number of SNPs selected: 359

```

# Read in the clumped model
clumped_model_final <- read.table(here("ProcessedData2", "training_indivs",
                                       "clumped_model_final.clumped"),
                                 header = TRUE)
nrow(clumped_model_final)

```

Number of risk SNPs: 198

```

# Read in the score input final
score_input_final <- read.table(here("ProcessedData2", "training_indivs",
                                       "score_input_final.txt"),
                               header = FALSE)
# logOR > 0: increased risk
positive_snps <- score_input_final[score_input_final[,3] > 0, ]
nrow(positive_snps)

# Save SNPs
write.table(positive_snps,
            here("Results", "positive_snps.txt"),
            quote = FALSE,
            row.names = FALSE,
            col.names = FALSE)

```

Number of protective SNPs: 160

```

# logOR < 0: decreased risk
negative_snps <- score_input_final[score_input_final[,3] < 0, ]
nrow(negative_snps)

# Save SNPs
write.table(negative_snps,

```

```

here("Results", "negative_snps.txt"),
quote = FALSE,
row.names = FALSE,
col.names = FALSE)

```

Adjusted PRS

To calculate the adjusted scores, compute the average effect of sex.

```

# Read in the training association results
assoc_model_final <- read.table(here("ProcessedData2", "training_indivs",
                                   "logistic_model_final.assoc.logistic"),
                               header = TRUE)

# Find common SNPs
common_snps_final <- intersect(assoc_model_final$SNP,
                               clumped_model_final$SNP)

# Calculated the average sex effect
sex_effect <- assoc_model_final %>%
  filter(TEST == "COV1") %>%
  mutate(logOR = log(OR)) %>%
  filter(SNP %in% common_snps_final)
meanlogOR <- mean(sex_effect$logOR)
meanlogOR

# Merge preds_final with phenotype data
pheno_test <- bfile_test$fam
preds_final <- merge(preds_final, pheno_test,
                    by.x = "IID", by.y = "member")
preds_final$SCORE_adj <- preds_final$SCORE + meanlogOR * preds_final$sex

# Calculate AUC for adjusted scores
roc_obj_final_adj <- roc(preds_final$PHENO, preds_final$SCORE_adj)
auc_val_final_adj <- auc(roc_obj_final_adj)
auc_val_final_adj

# Save roc object
saveRDS(roc_obj_final_adj,
        file = here("Results", "roc_obj_final_adjusted.rds"))

```

meanlogOR is 1.262464. Adjusted AUC is slightly better: 0.7606. (Unadjusted AUC is 0.724.)

```

# Save the adjusted predictions
write.table(preds_final,
            here("Results",
                "preds_final_adjusted.txt"),
            quote = FALSE,

```

```
row.names = FALSE,  
col.names = TRUE)
```

Model 2: Penalized Regressions

Use $\alpha = 1$ (lasso), 0.5 (elastic net), and 0 (ridge).

Load Data and Impute Based on Means

Read in the training dataset and testing dataset.

```
library(here)  
library(snpStats)  
library(glmnet)  
library(pROC)  
library(dplyr)  
library(doParallel)  
library(ggplot2)  
  
# Phenotype  
pheno <- read.csv(  
  here("RawData", "phenotype.csv"),  
  stringsAsFactors = FALSE  
)  
  
# Training dataset  
# Read in plink file  
bfile_train <- read.plink(here("ProcessedData2", "training_indivs.bed"),  
  here("ProcessedData2", "training_indivs.bim"),  
  here("ProcessedData2", "training_indivs.fam"))  
  
G_train <- bfile_train$genotypes  
X_train <- as(G_train, "numeric")  
  
# Testing dataset  
# Read in plink file  
bfile_test <- read.plink(here("ProcessedData2", "testing_indivs.bed"),  
  here("ProcessedData2", "testing_indivs.bim"),  
  here("ProcessedData2", "testing_indivs.fam"))  
  
G_test <- bfile_test$genotypes  
X_test <- as(G_test, "numeric")  
  
# Check whether X_test and X_train have the same SNPs  
all(colnames(X_test) == colnames(X_train)) # TRUE  
  
# Impute missing values with column means of X_train
```

```

for(j in seq_len(ncol(X_train))) {
  # Get the mean for the training set
  mean_X_train <- mean(X_train[,j], na.rm=TRUE)
  # Impute missing values in both training and testing sets
  X_train[is.na(X_train[,j]),j] <- mean_X_train
  X_test[is.na(X_test[,j]),j] <- mean_X_train
}

# Filter to only include individuals in the training data
pheno <- pheno %>%
  rename(IPF_status=phenotype, Sex=SEX) %>%
  mutate(Sex=ifelse(Sex=="Male",1,0), # Female as the reference
         IPF_status = factor(IPF_status, levels = c("control", "case")))
pheno_train <- pheno %>% filter(id %in% rownames(X_train))
pheno_test <- pheno %>% filter(id %in% rownames(X_test))

y_bin_train <- pheno_train$IPF_status
y_bin_test <- pheno_test$IPF_status
# Sanity check
table(pheno$Sex)
table(pheno$IPF_status)

```

Variable Selection

In this step, we use the top 2000 SNPs with the highest variance to reduce the dimensionality of the data. That's why Lauren's code runs faster.

```

# Select the best SNPs based on variance in the training data
snp_var <- apply(X_train, 2, var)
top_snps <- names(sort(snp_var, TRUE))[1:2000]

# Filter the training and testing data
X_small_train <- X_train[, top_snps, drop=FALSE]
X_small_test <- X_test[, top_snps, drop=FALSE]

```

Penalized Logistic Regression (with sex)

Let's train the model.

```

# Check the IDs and rownames match in the training set
all(pheno_train$id == rownames(X_small_train))

# Design matrix for training
X_design_train <- cbind(Sex = pheno_train$Sex, X_small_train)

# Parallel setup
set.seed(6655)

```

```

cl <- makeCluster(max(1, detectCores()-1)); registerDoParallel(cl)

alphas      <- c(1, 0.5, 0)
reported_snps <- c("rs12 345","rs23 456","rs34 567",
                  "rs45 678","rs56 789","rs67 890","rs78 901")
results     <- list()

for(a in alphas) {
  cvf <- cv.glmnet(
    x      = X_design_train,
    y      = y_bin_train,
    family = "binomial",
    alpha  = a,
    nfolds = 5, # 5-fold cross-validation
    type.measure = "auc",
    standardize = TRUE, # Standardize the predictors
    parallel = TRUE
  )

  lam <- cvf$lambda.min
  auc <- cvf$cvm[which(cvf$lambda==lam)]

  # Perform the final model fit with the selected lambda
  mdl <- glmnet(
    x      = X_design_train,
    y      = y_bin_train,
    family = "binomial",
    alpha  = a,
    lambda = lam,
    standardize = TRUE
  )
  coefs <- as.matrix(coef(mdl))

  results[[paste0("alpha=",a)]] <- list(
    lambda_min = lam,
    auc        = auc,
    nonzero    = sum(coefs[-1,]!=0),
    sex_coef   = coefs["Sex",1],
    snps_included = reported_snps %in% rownames(coefs)[coefs[,1]!=0],
    model = mdl
  )
}

stopCluster(cl)

# Print quick summary
for(nm in names(results)) {
  r <- results[[nm]]
  cat(nm,

```

```

    "|  $\lambda_{\min}$  =", format(r$lambda_min, digits=7),
    "| CV-AUC =", format(r$auc, digits=7),
    "| #nonzeros =", r$nonzero, "\n")
}

# Check the coefficients for sex
lapply(results, function(x) x$sex_coef)
lapply(results, function(x) exp(x$sex_coef))

```

Model Evaluation

Now, let's evaluate the model on the testing set (20%).

```

# Check whether X_test and X2 have the same SNPs
all(colnames(X_test) == colnames(X_train)) # TRUE

# Check whether the IDs and rownames match in the testing set
all(pheno_test$id == rownames(X_small_test))

# Design matrix for testing
X_design_test <- cbind(Sex = pheno_test$Sex, X_small_test)

roc_list <- list()

preds_list <- list()

# Loop through alphas and compute ROC
for (a in alphas) {
  mdl <- results[[paste0("alpha=", a)]]$model

  # Predict on the test set
  preds <- predict(mdl, newx = X_design_test,
    type = "response")

  # Compute ROC
  # Note: preds is a matrix; needs to be converted to a vector
  roc_obj <- roc(pheno_test$IPF_status, c(preds))

  # Save the ROC object
  roc_list[[paste0("alpha=", a)]] <- roc_obj

  # Save the preds
  preds_list[[paste0("alpha=", a)]] <- data.frame(
    id = rownames(X_design_test),
    preds = preds)
}

lapply(roc_list, function(x) x$auc)

```

```

# Save rds for later use
saveRDS(roc_list, here("Results", "Model2_roc_list.rds"))

# Save the predictions
saveRDS(preds_list, here("Results", "Model2_preds_list.rds"))

# Save model results
saveRDS(results, here("Results", "Model2_results.rds"))

```

Results

ROC Curves

```

# Read in the ROC objects
# 2 results from Model 1
roc_1_unadj <- readRDS(here("Results/roc_obj_final_unadjusted.rds"))
roc_1_adj <- readRDS(here("Results/roc_obj_final_adjusted.rds"))

# 3 results from Model 2
roc_2_list <- readRDS(here("Results/Model2_roc_list.rds"))
names(roc_2_list) <- c("Lasso", "Elastic Net", "Ridge")

# Combine all of them
roc_list <- list(
  "Model 1 Unadjusted" = roc_1_unadj,
  "Model 1 Adjusted" = roc_1_adj,
  "Model 2 Lasso" = roc_2_list[[1]],
  "Model 2 Elastic Net" = roc_2_list[[2]],
  "Model 2 Ridge" = roc_2_list[[3]]
)

plot_df <- do.call(rbind, lapply(names(roc_list), function(name) {
  roc_obj <- roc_list[[name]]
  coords_df <- coords(roc_obj, "all", ret = c("specificity", "sensitivity"))
  data.frame(
    specificity = coords_df[, "specificity"],
    sensitivity = coords_df[, "sensitivity"],
    Model = name
  )
}))

plot_df$Model <- factor(plot_df$Model, levels = c(
  "Model 1 Unadjusted",
  "Model 1 Adjusted",
  "Model 2 Lasso",
  "Model 2 Elastic Net",
  "Model 2 Ridge"
))

```

```

))

# Plot
ggplot(plot_df, aes(x = 1 - specificity, y = sensitivity, color = Model)) ♦
  geom_line(size = 1) ♦
  geom_abline(linetype = "dashed") ♦
  labs(
    x = "False Positive Rate (1 - specificity)",
    y = "True Positive Rate (sensitivity)",
    title = "ROC Curves Five Models"
  ) ♦
  theme_minimal()

# Save the plot
ggsave(
  here("Results/roc_curves.png"),
  width = 8,
  height = 6,
  dpi = 300
)

# AUC table
auc_result <- sapply(roc_list, function(roc_obj) {
  auc(roc_obj)
})

# Save the AUC table
write.csv(
  data.frame(Model = names(auc_result), AUC = auc_result),
  here("Results/auc_table.csv"),
  row.names = FALSE
)

```

Beeswarm Plots

```

# Read in PRS scores from Model 1
prs_1 <- read.table(here("Results/preds_final_adjusted.txt"), header = TRUE)
prs_df1 <- data.frame(
  id = rep(prs_1$IID, 2),
  PRS = c(prs_1$SCORE, prs_1$SCORE_adj),
  Model = rep(c("Model 1 Unadjusted", "Model 1 Adjusted"), each = nrow(prs_1))
)

# Read in PRS scores from Model 2
prs_2 <- readRDS(here("Results/Model2_preds_list.rds"))
names(prs_2) <- c("Lasso", "Elastic Net", "Ridge")
prs_df2 <- do.call(rbind, prs_2)
# Assign alpha levels
prs_df2$Model <- rep(paste0("Model 2 ", names(prs_2)),

```

```

      each = nrow(prs_df2) / 3)
colnames(prs_df2) <- c("id", "PRS", "Model")

# Combine the data frames
prs_df <- rbind(prs_df1, prs_df2)

# Read in the phenotype data
pheno <- read.csv(
  here("RawData", "phenotype.csv"),
  stringsAsFactors = FALSE
)

preds_df <- merge(prs_df, pheno, by = "id")

preds_df$Model <- factor(preds_df$Model, levels = c(
  "Model 1 Unadjusted",
  "Model 1 Adjusted",
  "Model 2 Lasso",
  "Model 2 Elastic Net",
  "Model 2 Ridge"
))

ggplot(preds_df, aes(x = phenotype, y = PRS)) ♦
  geom_beeswarm(aes(color = phenotype), size = 0.5, alpha = 0.5) ♦
  facet_wrap(~ Model, scales = "free_y", nrow = 1) ♦ # ← key change here
  labs(
    x = "Phenotype",
    y = "PRS Scores",
    title = "Beeswarm of Predicted Probabilities by Phenotype and Alpha"
  ) ♦
  theme_minimal() ♦
  theme(axis.text.x = element_text(angle = 90, vjust = 0.5, hjust = 1))
ggsave(
  here("Results/beeswarm_plot.png"),
  width = 12,
  height = 5,
  dpi = 300)

```

Euler Plot

```

reported_snps <- c("rs12 345","rs23 456","rs34 567",
  "rs45 678","rs56 789","rs67 890","rs78 901")

# Read in the positive and negative snps from Model 1
pos_SNPs <- read.table(
  here("Results/positive_snps.txt"),
  header = FALSE
)

```

```

neg_SNPs <- read.table(
  here("Results/negative_snps.txt"),
  header = FALSE
)

# Read model rds from Model 2
model_2 <- readRDS(here("Results/Model2_results.rds"))

lasso_md1 <- model_2`alpha=1`$model
elastic_md1 <- model_2`alpha=0.5`$model

# Get the non-zero coefficients
lasso_snps <- as.matrix(lasso_md1$beta)
lasso_pos <- rownames(lasso_snps)[lasso_snps > 0]
lasso_neg <- rownames(lasso_snps)[lasso_snps < 0]

elastic_snps <- as.matrix(elastic_md1$beta)
elastic_pos <- rownames(elastic_snps)[elastic_snps > 0]
elastic_neg <- rownames(elastic_snps)[elastic_snps < 0]

# Create a list of sets for significant SNPs
snps_list <- list(
  "Marginal" = c(pos_SNPs$V1, neg_SNPs$V1),
  "Lasso" = c(lasso_pos, lasso_neg)[-1],
  "Elastic Net" = c(elastic_pos, elastic_neg)[-1],
  "Reported" = reported_snps
)

# Create the Euler plot
fit <- euler(snps_list)
png(
  here("Results/euler_plot_all.png"),
  width = 1100,
  height = 600,
  res = 150, bg = "transparent"
)
plot(fit,
  fills = list(fill = c("skyblue", "orange", "green", "plum"), alpha = 0.5),
  labels = list(font = 2),
  quantities = TRUE,
  edges = TRUE)
dev.off)

```